Containers

and Docker - Part 2

Lecture 04 - Part 2

Academic Year 2025/2026

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste

Part 2 Overview

In this part, we will cover:

- Docker Networking: Bridge, host, overlay networks
- Docker Storage: Volumes, bind mounts, tmpfs
- Docker Compose: Multi-container applications
- Container Registries: Docker Hub, private registries
- Security: Best practices, rootless containers
- Container Runtimes: containerd, CRI-O, runc

Part 1

Docker Networking

Connecting containers

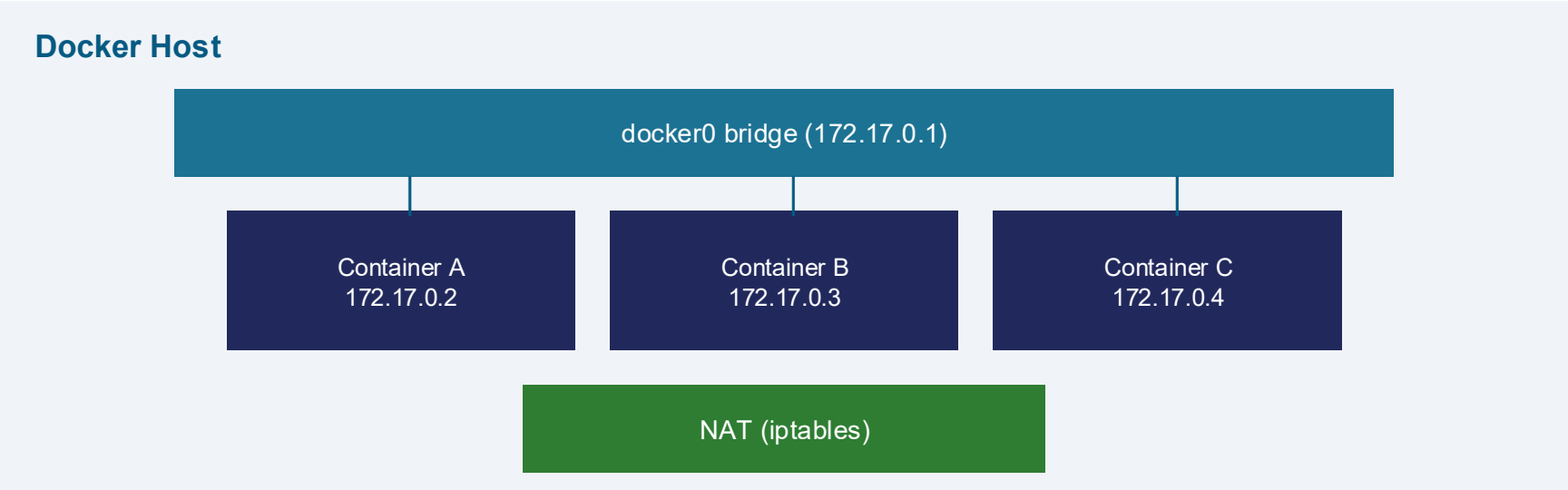
Docker Network Drivers

Driver	Use Case	Connectivity
bridge	Default; isolated network on single host	Containers on same bridge
host	Container shares host network stack	Same as host (no isolation)
none	No networking	Completely isolated
overlay	Multi-host networking (Swarm/K8s)	Containers across hosts
macvlan	Container gets MAC address on physical net	Direct to physical network

```
docker network ls                    # List networks
docker network create mynet          # Create bridge network
docker run --network mynet myapp     # Connect container
```

Bridge Network (Default)

Isolated network on a single Docker host



Containers can reach each other by IP; DNS works with user-defined bridges

Port Mapping

Expose container ports to the host

```
# Map host port 8080 to container port 80
docker run -p 8080:80 nginx

# Map to specific host interface
docker run -p 127.0.0.1:8080:80 nginx

# Random host port
docker run -p 80 nginx   # Check with docker port <container>
```

Syntax: -p [host_ip:]host_port:container_port[/protocol]

- Host port 8080 → Container port 80
- Traffic: Client → Host:8080 → iptables NAT → Container:80

Container DNS and Service Discovery

User-defined networks provide automatic DNS

```
# Create network and run containers
docker network create myapp
docker run -d --name db --network myapp postgres
docker run -d --name web --network myapp mywebapp

# Inside 'web' container:
$ ping db          # Resolves to db container IP
$ curl http://db:5432   # Connect by name
```

Key points:

- Default bridge: No DNS, use --link (deprecated) or IPs
- User-defined bridge: Automatic DNS by container name
- Container names become hostnames within the network

Part 2

Docker Storage

Persisting data

The Container Storage Problem

Container filesystem is ephemeral!
When container is removed, all data inside is lost.

Challenges:

- Database containers need persistent storage
- Container layer writes are slow (copy-on-write)
- Data should be shareable between containers
- Configuration files need to be injected

Docker Storage Options

Type	Description	Use Case
Volumes	Managed by Docker in /var/lib/docker/volumes	Databases, persistent data
Bind mounts	Map host path into container	Config files, dev (code sync)
tmpfs	In-memory filesystem	Secrets, temp data (no persist)

Volume
(Docker managed)

Bind Mount
(Host path)

tmpfs
(Memory only)

Volumes are preferred for production - portable and Docker-managed

Docker Volumes

Preferred mechanism for persistent data

```
# Create a volume
docker volume create pgdata

# Use volume in container
docker run -d --name postgres \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

# Volume persists after container removal
docker rm postgres    # Volume still exists
docker volume ls      # pgdata still there
```

Benefits: Managed by Docker, easy backup, works on all platforms, volume drivers for cloud storage

Bind Mounts

Map host directory/file directly into container

```
# Mount current directory for development
docker run -v $(pwd):/app -w /app node:18 npm start

# Mount config file (read-only)
docker run -v /etc/myapp/config.yml:/app/config.yml:ro myapp

# New syntax (more explicit)
docker run --mount type=bind,src=$(pwd),dst=/app myapp
```

Use cases:

- Development: Live code reloading
- Configuration: Inject config files
- ⚠️ Caution: Host path must exist; security implications

Part 3

Docker Compose

Multi-container applications

What is Docker Compose?

Define and run multi-container applications with a single YAML file

Instead of:

```
docker network create myapp
docker run -d --name db ...
docker run -d --name cache ...
docker run -d --name web ...
```

You write:

```
# docker-compose.yml
services:
  db: ...
  cache: ...
  web: ...
```

```
docker compose up -d      # Start everything
docker compose down       # Stop and remove
```

Docker Compose Example

```yaml
# docker-compose.yml
services:
  web:
    build: ./web
    ports:
      - "8080:80"
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgres://db:5432/myapp

  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_PASSWORD=secret

volumes:
  pgdata:
```

Compose creates network automatically; services discover each other by name

Docker Compose Commands

Command	Description
docker compose up -d	Create and start all services (detached)
docker compose down	Stop and remove containers, networks
docker compose ps	List running services
docker compose logs -f	Follow logs from all services
docker compose exec web bash	Execute command in running service
docker compose build	Build/rebuild service images
docker compose pull	Pull latest images
docker compose restart	Restart services

Tip: docker compose watch for auto-rebuild during development (Compose 2.22+)

Part 4

Container Registries

Storing and distributing images

Container Registries

Central repository for storing and distributing container images

Registry	URL	Notes
Docker Hub	docker.io	Default; free public, paid private
GitHub Container	ghcr.io	Integrated with GitHub Actions
Google Artifact	gcr.io / us-docker.pkg.dev	GCP native
Amazon ECR	<account>.dkr.ecr.<region>.amazonaws.com	AWS native
Azure ACR	<name>.azurecr.io	Azure native
Quay.io	quay.io	Red Hat; security scanning
Self-hosted	registry.example.com	Harbor, GitLab Registry

```
docker login ghcr.io      # Authenticate
docker push ghcr.io/user/myapp:v1    # Push image
```

Registry Workflow

Typical CI/CD workflow:

1. Developer pushes code to Git
2. CI pipeline builds Docker image
3. CI pushes image to registry (tagged with git SHA or version)
4. CD deploys by pulling specific image tag to production

Part 5

Container Security

Best practices and considerations

Container Security Considerations

Containers share the kernel - security requires attention

Kernel exploits:	Container escape via kernel vulnerability
Image vulnerabilities:	Outdated packages in base images
Running as root:	Container root = more capabilities than needed
Secrets in images:	Passwords/keys baked into layers
Unrestricted resources:	Container can exhaust host resources

Defense in depth: Apply multiple layers of security controls

Container Security Best Practices

1. **Use minimal base images**
   Alpine, distroless, scratch - smaller attack surface
2. **Run as non-root user**
   USER directive in Dockerfile; --user flag at runtime
3. **Scan images for vulnerabilities**
   Trivy, Snyk, Docker Scout, Clair
4. **Never store secrets in images**
   Use environment variables, Docker secrets, external vaults
5. **Set resource limits**
   --memory, --cpus to prevent resource exhaustion
6. **Use read-only filesystem where possible**
   --read-only flag prevents writes to container layer

Rootless Containers

Run Docker daemon and containers without root privileges

Traditional Docker: daemon runs as root (security risk)
Rootless Docker: daemon runs as regular user

Benefits:

- Container escape = user privileges only
- Multi-tenant safety
- Reduced attack surface

Alternatives:

- Podman (rootless by default)
- Singularity/Apptainer (HPC)
- User namespaces in Docker

Part 6

Container Runtimes

Beyond Docker

Container Runtime Ecosystem

Docker is not the only option - standards enable interoperability

Orchestration: Kubernetes, Docker Swarm

High-level Runtime: containerd, CRI-O

Low-level Runtime: runc, crun, gVisor, Kata

Linux Kernel: namespaces, cgroups, seccomp

OCI Standards: Image spec + Runtime spec = Interoperability

Container Runtimes Comparison

Runtime	Type	Use Case
runc	Low-level (OCI)	Default; standard Linux containers
crun	Low-level (OCI)	Faster, C implementation (Podman default)
gVisor (runsc)	Low-level (sandboxed)	Extra isolation; intercepts syscalls
Kata Containers	Low-level (VM)	MicroVM per container; strong isolation
containerd	High-level (CRI)	Kubernetes default; Docker uses it
CRI-O	High-level (CRI)	Kubernetes-native; minimal, secure
Podman	CLI + runtime	Docker-compatible; rootless by default

Kubernetes doesn't use Docker directly - it uses containerd or CRI-O

Containers for Scientific Computing

Special considerations for HPC environments

Challenges:

- HPC needs root-like access (MPI, InfiniBand)
- Shared filesystems (Lustre, GPFS)
- Scheduler integration (Slurm)
- GPU access

HPC Container Solutions:

- Apptainer (formerly Singularity)
- Sarus (CSCS)
- Shifter (NERSC)
- Enroot + Pyxis (NVIDIA)

```
Apptainer: Run Docker images on HPC clusters
$ apptainer pull docker://python:3.11
$ srun apptainer exec python_3.11.sif python script.py
```

Lecture 4: Key Takeaways

1. **Docker networking provides isolation and connectivity**
   Bridge networks with DNS; port mapping for external access

2. **Volumes persist data beyond container lifecycle**
   Prefer volumes over bind mounts for production

3. **Docker Compose simplifies multi-container apps**
   Single YAML file defines entire application stack

4. **Security requires defense in depth**
   Minimal images, non-root, scanning, resource limits

5. **Container ecosystem extends beyond Docker**
   OCI standards; containerd/CRI-O for K8s; Apptainer for HPC

Next: Lecture 5

Container Orchestration
Kubernetes Fundamentals

Pods • Services • Deployments • Architecture